

What to Cut? Predicting Unnecessary Methods in Agentic Code Generation

Kan Watanabe[†], Tatsuya Shirai[†], Yutaro Kashiwa[†], Hajimu Iida[†]

[†]Nara Institute of Science and Technology, Japan

{watanabe.kan.wi6,shirai.tatsuya.sp1,yutaro.kashiwa,iida}@naist.ac.jp

Abstract

Agentic Coding, powered by autonomous agents such as GitHub Copilot and Cursor, enables developers to generate code, tests, and pull requests from natural language instructions alone. While this accelerates implementation, it produces larger volumes of code per pull request, shifting the burden from implementers to reviewers. In practice, a notable portion of AI-generated code is eventually deleted during review, yet reviewers must still examine such code before deciding to remove it. No prior work has explored methods to help reviewers efficiently identify code that will be removed.

In this paper, we propose a prediction model that identifies functions likely to be deleted during PR review. Our results show that functions deleted for different reasons exhibit distinct characteristics, and our model achieves an AUC of 87.1%. These findings suggest that predictive approaches can help reviewers prioritize their efforts on essential code.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments**; **Automatic programming**; *Software evolution*; *Maintaining software*.

Keywords

Agentic Coding, Coding Agent, Pull Requests, Method Removal, Large Language Models

ACM Reference Format:

Kan Watanabe[†], Tatsuya Shirai[†], Yutaro Kashiwa[†], Hajimu Iida[†]. 2026. What to Cut? Predicting Unnecessary Methods in Agentic Code Generation. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793591>

1 Introduction

AI-assisted coding has become standard practice in modern software development. The integration of Large Language Models (LLMs) into development workflows has moved from experimental adoption to everyday use. Agentic Coding, which leverages autonomous coding agents such as GitHub Copilot [10] and Cursor [3], exemplifies this shift. Developers now delegate complex tasks, including code implementation, testing, and pull request creation, to AI agents guided solely by natural language instructions.

While this shift promises substantial gains in development efficiency [16, 18], it introduces a fundamental trade-off. Agentic Coding reduces the burden on implementers but increases the burden on reviewers. Concerns regarding the quality, maintainability, and redundancy of AI-generated code are critical issues. For instance, Watanabe *et al.* [23] revealed that code generated by AI agents tends to be larger than human-written code. Furthermore, Imai [11] showed that AI-assisted development results in more deleted code lines compared to human pair programming. These findings suggest that while Agentic Coding excels at generation, it also tends to produce excessive and unnecessary code.

Although Agentic Coding significantly reduces implementation time, the verification and review of generated code still rely on humans. The increased volume of code demands greater reviewing effort, creating a new bottleneck in the development process. Previous studies [17] suggest that pull requests generated by AI are likely to contain redundant methods that will eventually be deemed unnecessary, and scrutinizing all of these imposes a heavy cognitive load on human reviewers.

To the best of our knowledge, no studies have investigated the characteristics of AI-generated methods that are subsequently deleted. Predicting “methods that will be deleted in the revisions” would allow reviewers to prioritize their attention on essential code, thereby significantly reducing their cognitive burden. In this study, we track the change history of code generated by Agentic Coding to quantitatively characterize (e.g., lines of code, complexity, dependencies) methods deleted during review, from pull request creation to merging. We also construct a model to predict methods with a high probability of future deletion at the time of pull request creation and evaluate its effectiveness.

Replication Packages: To facilitate further studies, we provide the data and programs used in our replication package [22].

2 Data Collection

2.1 Dataset

We collect methods generated by agents at the time of PR creation and analyze whether they are deleted by the time the PRs are merged. To collect PRs created by agents (Agentic-PRs), we use the AIDev dataset [12], which contains 33,596 PRs created by five major AI agents.

We first extracted PRs generated by agents by querying the `pull_request` table in the dataset. We then filtered for PRs containing changes in Python code because the refactoring detection tool we use later supports only Python. We further filtered for merged pull requests only, as this allows us to determine whether each method was ultimately needed for production. This filtering resulted in 1,664 PRs from 197 projects. From these PRs, we



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR 2026, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/10.1145/3793302.3793591>

Table 1: Detailed Definitions of Extracted Method Features

Category	Feature Name	Description
Size	code_loc	Number of lines excluding comments, docstrings, and blank lines.
	char_length	Number of characters in the method.
	tokens	Number of tokens extracted by Python’s tokenize module excluding comments, indentations, and newlines.
	docstring_words	Number of words in the method’s docstring (0 if no docstring exists).
	method_name_words	Number of words in the method name, split by snake_case and CamelCase naming conventions.
Method Type	is_getter	Boolean indicating if the name starts with get (case-insensitive regex match).
	is_setter	Boolean indicating if the name starts with set (case-insensitive regex match).
	is_IsHas	Boolean indicating if the name starts with is or has (case-insensitive regex match).
	is_test	Boolean indicating if "test" is included in the method name (case-insensitive).
	is_in_test_code	Boolean indicating if "test" is included in the file path or method name.
	is_private	Boolean indicating if the name starts with _ (excluding dunder methods).
	is_dunder_method	Boolean indicating if the method is a dunder method (name surrounded by "__", e.g., __init__).
Contents	has_return	Boolean indicating if the method contains a return statement with a value.
	param_count	Number of parameters (positional, keyword, *args, **kwargs).
	number_of_variable	Number of variables in the method, calculated with ast.Assign and ast.AnnAssign.
	call_expression_count	Number of function call nodes (ast.Call) within the method body.
	number_of_print	Number of function calls explicitly invoking print.
	comment_ratio	Ratio of comment lines (lines starting with #) to the total physical lines of code.
	cyclomatic_complexity	Cyclomatic complexity calculated via the radon library (default: 1 if calculation fails).
	halstead_volume	Halstead Volume calculated via the radon library (default: 0.0 if calculation fails).
	max_nesting_depth	Maximum nesting depth of control structures (If, For, While, Try, With) in the AST.
	uses_try_except	Boolean indicating the presence of exception handling blocks (try-except).
	uses_constants	Boolean indicating if the method uses constant-style variables.

extracted methods and filtered them to ensure each method was unique, identified by its file path, class name, and method name. Next, we identified three revisions from each collected pull request: the base revision (when a branch was created for the PR), the PR creation revision (when the PR was opened), and the merge revision (when the PR was integrated into the target branch, e.g., main).

2.2 Method Identification

First, we identified methods in the changed Python files of each PR. We constructed Abstract Syntax Trees (ASTs) using Python’s ast module [19] to parse all methods at each revision. Specifically, we traversed FunctionDef and AsyncFunctionDef nodes, extracting class methods, top-level functions, and nested functions. Each method was uniquely identified by the combination of file path, class name, and method name.

We then identified the methods added between the base revision and the PR creation revision by comparing the ASTs of the two revisions. Similarly, we identified the methods deleted between the PR creation revision and the merge revision. Methods present in the PR creation revision but absent in the base revision were considered added, while methods present in the PR creation revision but absent in the merge revision were considered deleted.

However, between these revisions, methods, classes, and directories may have been renamed or moved. To address this, we detected refactoring operations using ActRef [20], a state-of-the-art refactoring detection tool for Python. It detects 15 types of refactoring operations, such as “Rename Method”, “Move Method”, “Inline Method”, and “Extract Method”. Previous studies reported that the tool achieves 78% precision and 91% recall [21].

We applied this tool to all intermediate revisions in chronological order, both between the base revision and the PR creation revision and between the PR creation revision and the merge revision. By executing ActRef for each adjacent revision pair (C_i and C_{i+1}) and tracking changes in method names, class names, and file paths step by step, we accurately determined the survival status of methods that underwent multiple refactorings or complex changes. When we detected rename or move refactoring, we did not classify these methods as added or deleted.

Based on this analysis, we label the added methods that were deleted between the PR creation revision and the merge revision as “Deleted Methods”, while methods that persisted until the merge revision (including those that were renamed or moved) are labeled as “Survived Methods”.

2.3 Metrics Measurement

To characterize the properties of methods generated by agents, we extracted 23 features from each method at the PR creation revision. Table 1 presents the detailed definitions of these features. These features are organized into three categories: Size, Method Type, and Contents, based on metrics commonly used in code quality analysis [6]. Specifically, we used the radon library [15] to calculate cyclomatic complexity and Halstead volume. We extracted other features by analyzing the AST, source code tokens, and method signatures using Python’s standard ast, tokenize, and re modules.

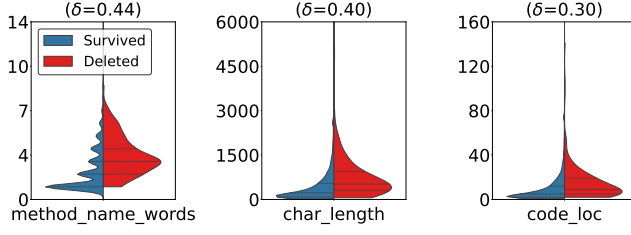


Figure 1: Feature difference (Top-3)

3 Research Questions

RQ₁: How often are agent-generated methods deleted, and what characterizes them?

Approach. We calculate the proportion of methods introduced at PR creation that are deleted by the time of merging. To identify characteristics that distinguish deleted methods from survived ones, we compare the distributions of the 23 extracted features between the two groups. We apply the Mann-Whitney U test to determine statistical significance and calculate Cliff’s delta effect size to quantify the magnitude of these differences.

Results. We identified 12,343 methods added at PR creation. Of these, 97.4% survived and 2.6% were deleted (*i.e.*, 323 methods). Among the deleted methods, 1.0% were deleted due to file deletions and 1.6% were deleted independently. While this percentage appears small, most PRs (75.6%) contain no revisions. Focusing on PRs with revisions (397 PRs containing 3,257 methods), 9.9% of methods are deleted (6.0% at the method level and 3.9% due to file deletions). Note that the following analysis, including RQ2, excludes file-level deletions because these may not reflect the characteristics of the deleted methods themselves.

We then compared the metrics of surviving and deleted methods. Out of 23 metrics, we found statistically significant differences in 18 metrics. Figure 1 shows the top 3 features with the highest effect size, comparing “Survived” and “Deleted” methods. Additional figures are available in the online appendix.¹

The feature `method_name_words` had the highest effect size, indicating that deleted methods tend to have longer method names. For `char_length`, deleted methods contained more characters. As shown in the figure, `code_loc` indicates that deleted methods tend to contain more row of code.

Answer to RQ1. 9.9% of methods are deleted in PRs that require revisions: 6.0% are deleted at the method level and 3.9% are deleted at the file level.

RQ₂: How accurately can method deletions be predicted?

Approach. We develop a binary classification model to identify methods added in PRs that will be deleted before the PRs are merged, using the features shown in Table 1. We use the Random Forest (RF) algorithm because it handles heterogeneous features well, provides

interpretable feature importance, and has shown strong performance in software engineering classification tasks [9]. To build the model with imbalanced labels in the dataset, we undersample the majority class (“Survived”) to match the minority class.

Baselines. We establish two baselines for comparison: random prediction and a commercial LLM. For random prediction, we assign binary labels based on the ratio of labels in the dataset. For the LLM baseline, we provide the source code of the method as input to GPT-4o. A previous study [7] shows that LLMs can be useful for detecting dead code. Specifically, we use the following prompt: “Predict if the following method will be deleted (0) or survived (1) in the code review. Briefly explain the reason. The output should be in JSON.”

Evaluation. To evaluate model performance, we conducted 10-fold cross-validation and computed the median of performance metrics across folds. We used Accuracy, Precision, Recall, F1-score, and AUC as evaluation metrics. Additionally, we analyzed which features contribute to prediction by calculating the median feature importance from Random Forest.

Table 2: Prediction results

evaluation	Random	GPT-4o	Proposed
AUC	0.500	0.621	0.871
Accuracy	0.498	0.836	0.735
F1-Score	0.029	0.004	0.092
Precision	0.015	0.002	0.049
Recall	0.475	0.026	0.825

Results. Table 2 shows the prediction performance of our model, the LLM, and the random approach. The proposed method achieved an AUC of 87.1%, exceeding both GPT-4o and the random baseline. GPT-4o exhibited higher Accuracy (83.6%) than the proposed method and the random baseline. However, GPT-4o’s Recall was extremely low (2.6%), indicating a tendency to classify most instances as the majority class, “Survived”. In contrast, although the proposed method had low Precision (4.9%), its Recall was high at 82.5%, demonstrating its ability to detect a wide range of methods that are actually deleted.

Figure 2 shows an example of a method that was actually deleted. While our proposed approach correctly predicted its deletion, GPT-4o misclassified it as “Survived”. GPT-4o justified its prediction by citing the code’s readability and robustness. Specifically, the model praised the code’s adherence to programming best practices, including type hints, side-effect avoidance using `.copy()`, and clear naming conventions, concluding that it was well-maintained code likely to be preserved. However, this code was deleted in reality. This instance suggests that LLMs conflate local code quality with system-level necessity.

```

1 def transform_container_instances(instances: List[Dict[str, Any]], region:
2     str) -> List[Dict[str, Any]]:
3     transformed: List[Dict[str, Any]] = []
4     for inst in instances:
5         i = inst.copy()
6         i["registeredAt"] = dict_date_to_epoch(i, "registeredAt")
7         i["Region"] = region
8         transformed.append(i)
9     return transformed

```

Figure 2: Misclassified Example by GPT.

¹http://github.com/kan0803/predict-deletion-method/blob/main/RQ1%20violine_all.pdf

Table 3: Feature importance rate (Top-10)

Features	Median	Std
method_name_words	0.16	0.015
char_length	0.14	0.005
tokens	0.09	0.006
code_loc	0.07	0.002
call_expression_count	0.07	0.003
docstring_words	0.06	0.003
comment_ratio	0.06	0.007
halstead_volume	0.05	0.005
param_count	0.05	0.005
number_of_variable	0.05	0.004

Table 3 summarizes the top 10 feature importance values obtained from the Random Forest model. The top-ranked features, `method_name_words`, `char_length`, and `tokens`, are size-related. However, `method_name_words` exhibits the highest standard deviation (0.009), indicating variability in its predictive contribution across iterations. This suggests that long variable names do not deterministically lead to deletion.

The contents metrics `call_expression_count`, `halstead_volume`, and `param_count` also show relatively higher importance. Interestingly, method type features such as `is_private`, `is_IsHas`, and `is_getter` showed relatively low importance and did not appear in the top 10 features. This suggests that these features provided little utility in predicting the target class. This implies that deletion prediction cannot rely on simple method type classifications.

Answer to RQ2. The proposed model achieved high predictive performance with an AUC of 87.1%, outperforming GPT-4o.

4 Related Work

While agentic coding is expected to accelerate development through automated code generation, empirical studies suggest it can also impose a significant burden on developers. In particular, the time required to review AI-generated code has emerged as a critical concern [5, 16]. Mozannar *et al.* [16] conducted a qualitative study of programmer activities and reported that “verifying suggestions” accounts for 22.4% of total session time. Becker *et al.* [5] studied the impact of AI tools and found that their use actually increased development time by 19%. They attributed this increase to the time spent reviewing and correcting the generated code.

The quality of AI-generated code is also a subject of research [1, 8, 11, 17, 23]. Abbassi *et al.* [1] systematically classified inefficiencies in AI-generated code, revealing that about 90% of the generated output contained some form of inefficiency. Cotroneo *et al.* [8] identified a trend where AI-generated code frequently includes redundant or unnecessary operations. Imai [11] demonstrated that while AI increases the volume of code produced, a significant portion of those lines is subsequently deleted in later stages of the development process.

Several studies have also proposed methods to detect such redundant or suboptimal code in AI-generated output [2, 13, 14, 24]. Liu *et al.* [13] proposed DeRep, a tool designed to detect “repetitions” in

LLM-generated code. Zhang *et al.* [24] evaluated the effectiveness of LLMs in detecting code clones and found that LLMs perform better at identifying clones within AI-generated code than in human-written code. Mahmoudi *et al.* [14] introduced SpecDetect4AI for detecting AI-specific code smells, achieving 89% precision and recall while being 13–129 times faster than conventional methods.

Despite these advances in identifying reviewer burdens and code quality issues, how often developers delete methods and what characteristics lead to deletion in practice remain unclear.

5 Threats to Validity

Threats to internal validity: To accurately label methods as “Survived” or Deleted,” we rely on ActRef, a Python-based refactoring detection tool. While the tool shows strong performance compared with other refactoring detection tools for Python [4], we cannot entirely eliminate misclassification risks. This noise may affect label quality and introduce bias into model evaluation.

Threats to construct validity: Our definition of “Deleted” methods is limited to the review period, from pull request creation to merging. Methods deleted after merging are not captured. Additionally, deletion does not always indicate poor code quality; methods may be removed due to requirement changes, design decisions, or consolidation with existing functionality. The features we extracted, such as cyclomatic complexity and lines of code, may not fully capture all factors influencing deletion decisions.

Threats to external validity: This study exclusively targeted Python projects due to constraints of the refactoring detection tool and dataset. Python’s syntax and coding conventions may differ significantly from statically typed languages such as Java or C++. Certain features, such as `is_dunder_method`, rely on Python-specific specifications. Consequently, our findings and prediction model may not generalize to other programming languages.

6 Conclusion and Future work

To address the decline in code maintainability and the increased review load associated with the proliferation of Agentic Coding tools such as GitHub Copilot and Cursor, this study empirically investigated the characteristics of AI-generated methods deleted during code review and built a prediction model to identify methods likely to be deleted at the time of pull request creation.

Our results reveal that 9.9% of methods created by Agentic Coding are deleted in environments where method deletion occurs. Furthermore, the deleted methods tended to exhibit redundancy across their function names, code content, and documentation. Finally, our prediction model demonstrated high classification performance, achieving an average AUC of 87.1%.

In future work, we plan to extend our approach by incorporating additional metrics that detect redundant methods within the existing code base, such as semantic similarity with other methods and duplicate functionality detection, to further improve prediction accuracy and support reviewers in identifying unnecessary code.

Acknowledgments

This work was supported by JSPS KAKENHI (JP24K02921, JP25K21359) and JST PRESTO (JPMJPR22P3), ASPIRE (JPMJAP2415), and AIP Accelerated Program (JPMJCR25U7).

References

- [1] Altaf Allah Abbassi, Léuson M. P. da Silva, Amin Nikanjam, and Foutse Khomh. 2025. A Taxonomy of Inefficiencies in LLM-Generated Python Code. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution, ICSME 2025*. IEEE, 393–404.
- [2] Ajmain Inqiad Alam, Palash Ranjan Roy, Farouq Al-Omari, Chanchal K. Roy, Banani Roy, and Kevin A. Schneider. 2025. Are Classical Clone Detectors Good Enough for the AI Era?. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution, ICSME 2025*. 295–307.
- [3] Anysphere. 2025. Cursor Documentation. <https://docs.cursor.com/> Accessed: December 31st, 2025.
- [4] Hassan Atwi, Bin Lin, Nikolaos Tsantalis, Yutaro Kashiwa, Yasutaka Kamei, Naoyasu Ubayashi, Gabriele Bavota, and Michele Lanza. 2021. PYREF: Refactoring Detection in Python Projects. In *Proceedings of the 21st IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2021*. 136–141.
- [5] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *CoRR* abs/2507.09089 (2025).
- [6] Raymond P. L. Buse and Westley Weimer. 2010. Learning a Metric for Code Readability. *IEEE Transactions on Software Engineering, TSE* 36, 4 (2010), 546–558.
- [7] Minyu Chen, Guoqiang Li, Ling-I Wu, and Ruibang Liu. 2025. DCE-LLM: Dead Code Elimination with Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers*. 9942–9955.
- [8] Domenico Cotroneo, Cristina Improta, and Pietro Liguori. 2025. Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity. In *Proceedings of the 36th IEEE International Symposium on Software Reliability Engineering, ISSRE 2025*. IEEE, 252–263.
- [9] Seema Dewangan, Rajwant Singh Rao, Alok Mishra, and Manjari Gupta. 2021. A Novel Approach for Code Smell Detection: An Empirical Study. *IEEE Access* 9 (2021), 162869–162883.
- [10] GitHub, Inc. 2025. GitHub Copilot documentation. <https://docs.github.com/en/copilot> Accessed: December 31st, 2025.
- [11] Saki Imai. 2022. Is GitHub Copilot a Substitute for Human Pair-programming? An Empirical Study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2022*. ACM/IEEE, 319–321.
- [12] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Team-mates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *CoRR* abs/2507.15003 (2025).
- [13] Mingwei Liu, Juntao Li, Ying Wang, Xueying Du, Zuoyu Ou, Qiuyuan Chen, Bingxu An, Zhao Wei, Yong Xu, Fangming Zou, Xin Peng, and Yiling Lou. 2025. Code Copycat Conundrum: Demystifying Repetition in LLM-based Code Generation. *CoRR* abs/2504.12608 (2025).
- [14] Brahim Mahmoudi, Naouel Moha, Quentin Stievenert, and Florent Avellaneda. 2025. AI-Specific Code Smells: From Specification to Detection. *CoRR* abs/2509.20491 (2025).
- [15] Lacchia Michele. 2025. *Radon: Code Metrics in Python*. <https://github.com/rubik/radon>
- [16] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, CHI 2024*. ACM, 142:1–142:16.
- [17] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. 2025. Investigating The Smells of LLM Generated Code. *CoRR* abs/2510.03029 (2025).
- [18] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *CoRR* abs/2302.06590 (2023).
- [19] Python Software Foundation. 2025. *ast — Abstract Syntax Tree*. Python Software Foundation.
- [20] Siqi Wang, Xing Hu, Xin Xia, and Xinyu Wang. 2025. <https://figshare.com/s/984c7a39266137e29c37> Accessed: December 31st, 2025.
- [21] Siqi Wang, Xing Hu, Xin Xia, and Xinyu Wang. 2025. ActRef: Enhancing the Understanding of Python Code Refactoring with Action-Based Analysis. *CoRR* abs/2505.06553 (2025).
- [22] Kan Watanabe. 2025. Replication package for: What to Cut? Predicting Unnecessary Methods in Agentic Code Generation. GitHub repository, <https://github.com/kan0803/predict-deletion-method>. Accessed: 2026-01-26.
- [23] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *CoRR* abs/2509.14745 (2025).
- [24] Zixian Zhang and Takfarinas Saber. 2024. Assessing the Code Clone Detection Capability of Large Language Models. *CoRR* abs/2407.02402 (2024).